

SISTEMAS DISTRIBUIDOS I

(TA050) CURSO PABLO D. ROCA

Trabajo Práctico Diseño

Coffee Shop Analysis

16 de septiembre de 2025

Sebastian Mauricio Vintoñuke	106063
Marcelo Agustin Origoni	109903
Agustin Pelliciarì	108172

1. Alcance

El sistema tiene como propósito procesar y analizar información de ventas en una cadena de cafés en Malasia.

A través de un diseño distribuido, busca obtener métricas a partir de datos transaccionales, de clientes, tiendas y productos.

Alcance funcional, permite obtener:

- Transacciones realizadas entre 2024 y 2025 entre las 06:00 AM y las 11:00 PM con monto total mayor a 15.
- Nombre del producto más vendido y nombre del producto que más ganancias ha generado, para cada mes en 2024 y 2025.
- TPV (Total Payment Value) por cada semestre en 2024 y 2025, para cada sucursal, para transacciones realizadas entre las 06:00 AM y las 11:00 PM.
- Fecha de cumpleaños de los 3 clientes que han hecho más compras entre 2024 y 2025, para cada sucursal.

Alcance no funcional:

- Optimizado para entornos multicomputadoras.
- Soporta el incremento de los elementos de cómputo para escalar los volúmenes de información a procesar.
- Usa un Middleware para abstraer la comunicación basada en grupos.
- Soportar una única ejecución del procesamiento y provee graceful quit frente a señales SIG-TERM.

2. Arquitectura de Software

El sistema se basa en una arquitectura mixta. Adopta un modelo cliente-servidor para gestionar la interacción con los usuarios. Internamente, está conformado por múltiples componentes de cómputo que trabajan en conjunto, lo que permite atender los requerimientos de escalabilidad y aprovechar las ventajas de los entornos multicómputo.

2.1. Interacción con el cliente

La comunicación entre el cliente y el sistema se basa en un modelo cliente-servidor. El cliente envía una consulta en la que incluye los archivos CSV a procesar. El sistema recibe esta petición y responde con un requestId, un identificador único asociado a la consulta.

Más adelante, el cliente utiliza ese requestId para consultar el estado de su solicitud. Esta operación es bloqueante, ya que el cliente debe esperar hasta que el procesamiento haya finalizado para poder acceder a los resultados.

En conjunto, la interacción define un **protocolo de comunicación asíncrono**, estructurado en dos intercambios de tipo request-reply sincrónicos: el primero para registrar la consulta y obtener el identificador, y el segundo para recuperar los resultados una vez estén disponibles.

2.2. Interacción entre nodos

La interacción principal del sistema se establece entre el Dispatcher y los nodos de ejecución, responsables de llevar a cabo las distintas etapas de procesamiento que conforman los pipelines.

Esta coordinación se realiza a través de un **Message-Oriented Middleware (MOM)**, centralizado y basado en un modelo de colas asíncronas de mensajes. En este esquema, cada nodo tiene una cola de entrada y despacha sus resultados a una siguiente cola, al finalizar, se consolidan los resultados y se envían al almacenamiento persistente.

3. Vista Lógica

3.1. Entidades del dominio del negocio

Menu Item: Representa los productos del menú disponibles en cada tienda.

Store: Sucursales donde se realizan las transacciones.

Transaction Item: Relación entre un producto y una transacción específica, con cantidad y precio unitario.

Transaction: Operación de compra realizada por un cliente en una tienda en una fecha y hora determinada.

User: Cliente que interactúa con la cadena de cafés.

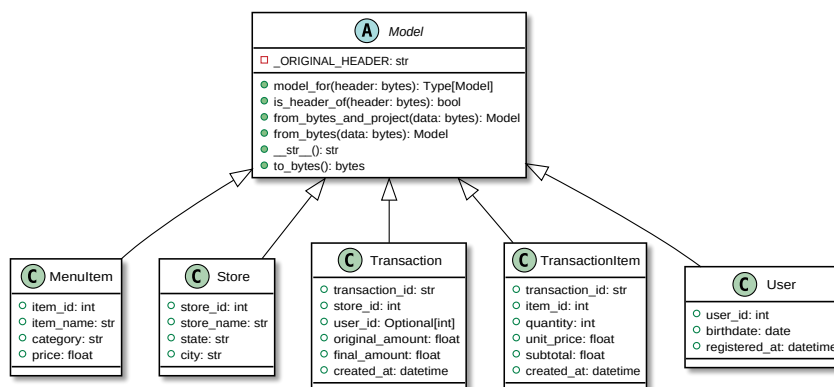


Figura 1: Diagrama de clases de los modelos de datos iniciales

Query 1: Transacciones válidas por período horario y monto mínimo

Query 2 Best Selling: Producto más vendido por mes

Query 2 Most Profit: Producto con mayor ganancia por mes

Query 3: TPV (Total Payment Value) por semestre y sucursal

Query 4: Clientes más frecuentes por sucursal

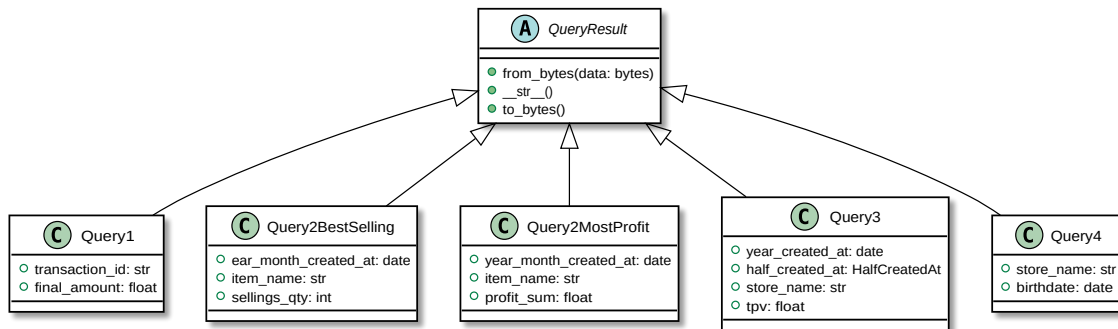


Figura 2: Diagrama de clases de los modelos de datos resultado

3.2. Entidades del dominio del sistema

Servidor: Punto de entrada del sistema. Recibe a los clientes y les asigna un dispatcher para que suban sus archivos o recibe un clientID y les informa su Data Storage asignado.

Dispatcher: Responsable de recibir los archivos, validar y planificar la ejecución de las consultas. Divide el trabajo en tareas y las envía a los distintos nodos de procesamiento.

Middleware (MOM): Componente centralizado de mensajería orientada a colas. Permite la comunicación asíncrona entre nodos y asegura la tolerancia a fallos.

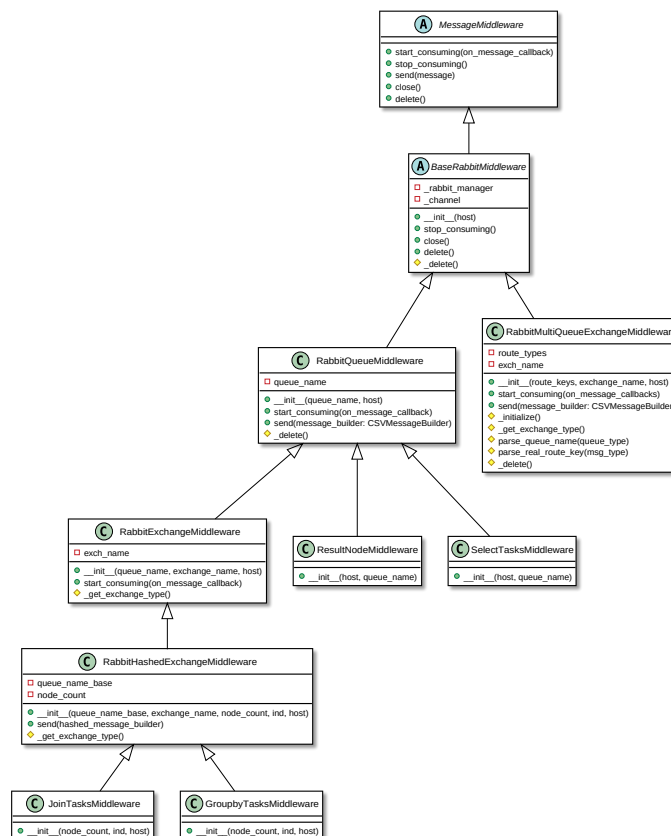


Figura 3: Diagrama de clases de la jerarquía del middleware

Select Node: Nodo encargado de aplicar filtros (por fecha, hora y monto) para reducir el

volumen de datos antes de etapas más complejas.

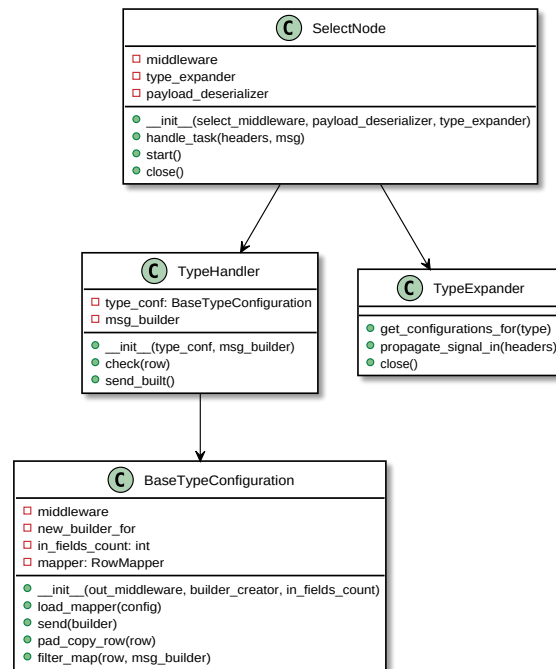


Figura 4: Diagrama de clases del nodo de selección

Group Node: Nodo que agrupa la información según distintos criterios (por ejemplo, por producto, por cliente o por sucursal) y calcula métricas agregadas.

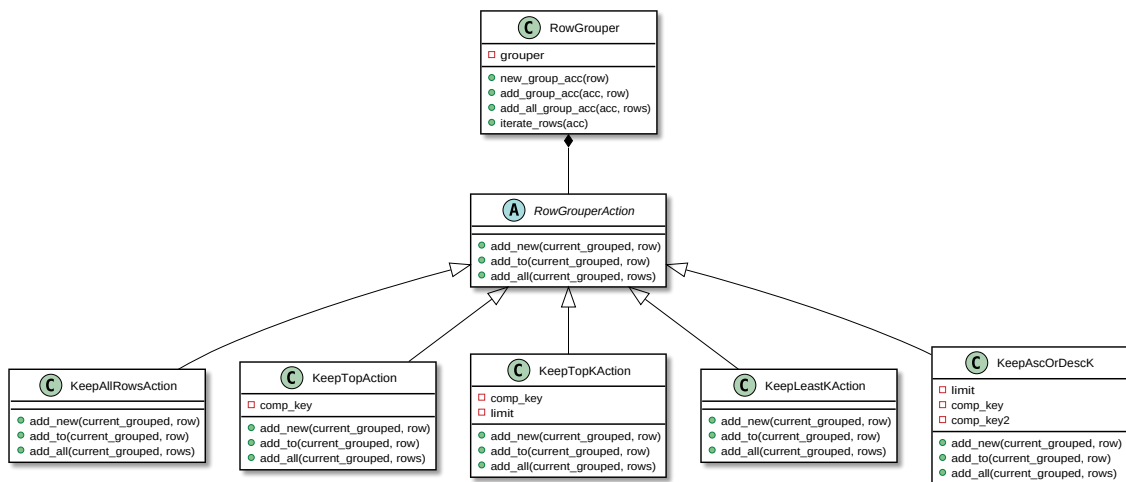


Figura 5: Diagrama de clases del nodo de agrupamiento

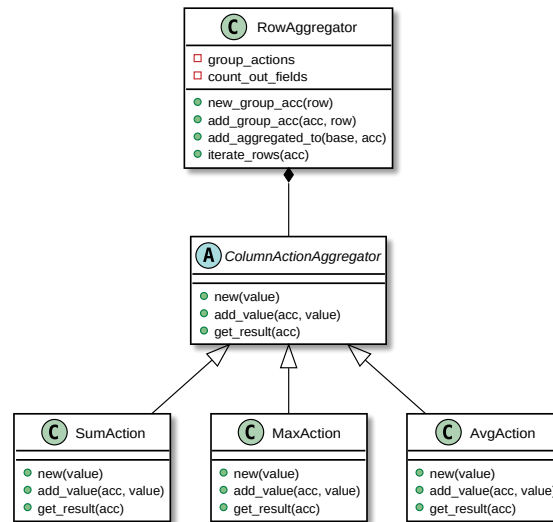


Figura 6: Diagrama de clases del nodo de agregación

Join Node: Nodo que une los resultados parciales con otras entidades del dominio (ejemplo: asociar item_id con Menu Item).

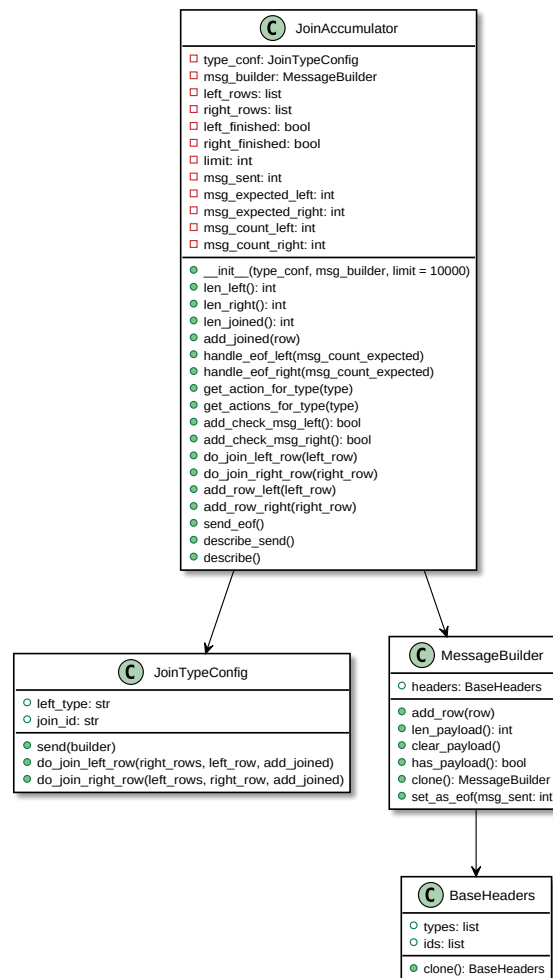


Figura 7: Diagrama de clases del nodo de unión

Result Node: Consolida los resultados de las consultas, mapea los resultados finales al formato deseado y verifica su finalización.

State Storage: Almacenamiento intermedio utilizado por partes del sistema que requieren mantener estado en caso de fallo.

- **Dispatcher:** Se almacenan los usuarios que están siendo atendidos hasta que las consultas son completamente recibidas, en caso de fallo, al reincorporarse se propagan los paquetes de abort asociados a estas consultas corruptas.
- **Group/Join/Result Node:** Se almacenan los resultados parciales acumulados, en caso de fallo, al reincorporarse se continúa el procesamiento desde el último estado.

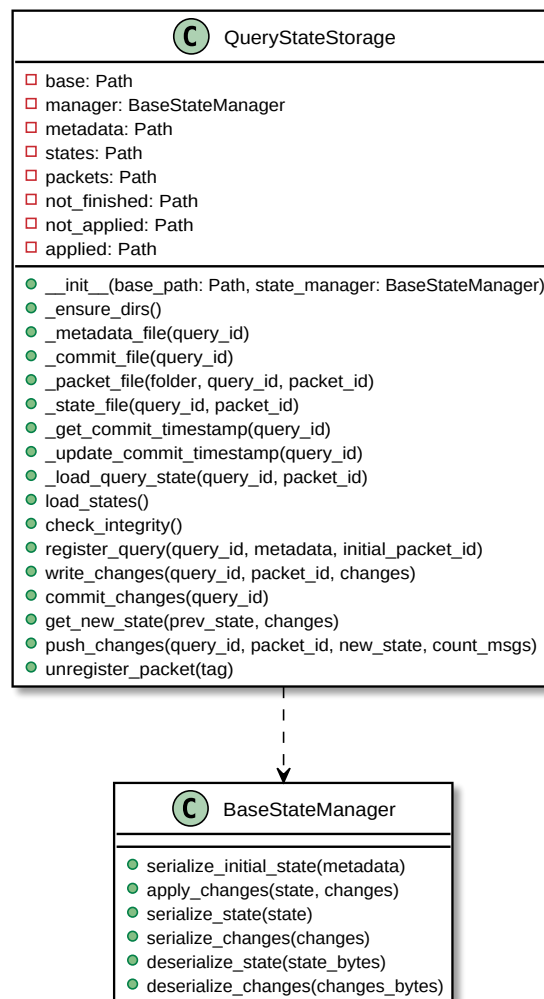


Figura 8: Diagrama de clases del State Storage

Result Storage: Repositorio persistente donde se guardan los resultados finales de cada consulta, asociados al cliente_id.

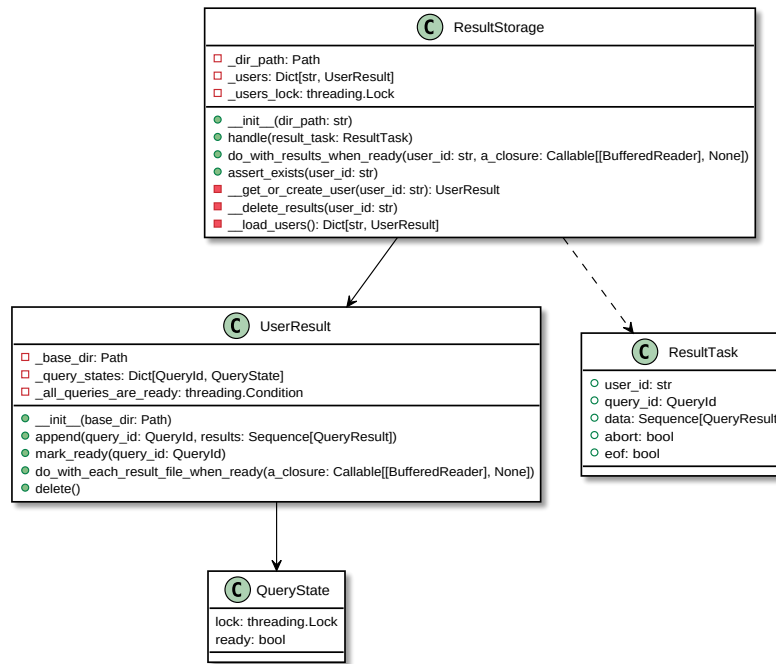


Figura 9: Diagrama de clases del almacenamiento de resultados

Supervisor: Componente encargado de monitorear el estado de los componentes del sistema mediante un algoritmo de heartbeat y reiniciarlos ante fallos. El propio supervisor está replicado en múltiples instancias para garantizar disponibilidad. La coordinación entre instancias se resuelve mediante un algoritmo de elección líder.

Task Queue: Cola de mensajes que intermedia entre los distintos nodos y asegura la distribución ordenada de las tareas de procesamiento.

3.3. Entidades de comunicación

Client Protocol: Gestiona la comunicación desde el cliente hacia el servidor o el dispatcher. Permite enviar los archivos CSV de entrada y recibir las direcciones de conexión asignadas. Es responsable de empaquetar y transmitir los datos de forma segura mediante los protocolos de bajo nivel.

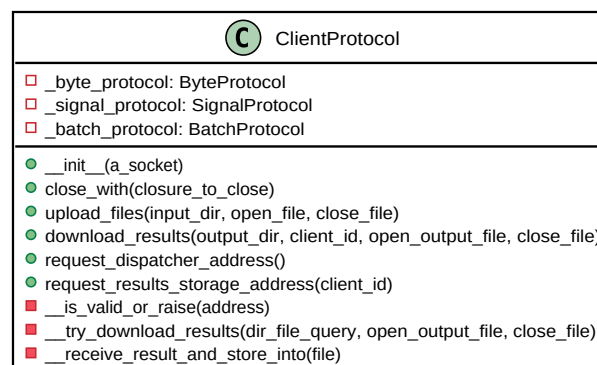


Figura 10: Diagrama de clase del protocolo del cliente

Server Protocol: Encapsula la lógica de comunicación entre el servidor y los clientes, distri-

buye la carga de clientes sobre el sistema, asigna direcciones de Dispatchers o Result Storages.

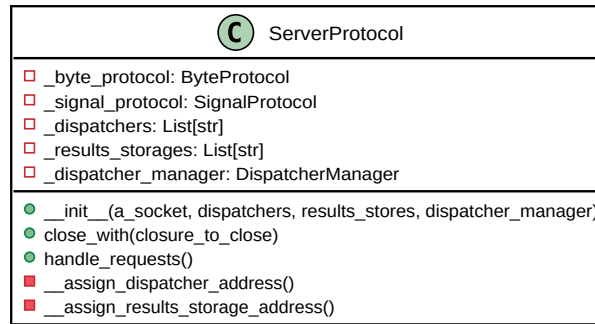


Figura 11: Diagrama de clase del protocolo del servidor

Dispatcher Protocol: Define la comunicación entre el cliente y el dispatcher. Se encarga de recibir los lotes de datos, identificar los modelos de entrada y distribuir las tareas a los nodos correspondientes (*Select*, *Join*, etc.).

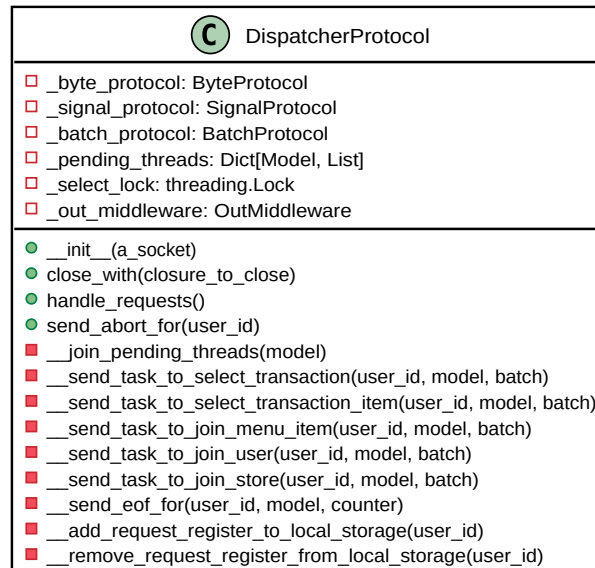


Figura 12: Diagrama de clase del protocolo del dispatcher

Result Protocol: Protocolo responsable de la comunicación entre el cliente y el storage de resultados. Gestiona la notificación de la finalización de consultas y el envío de los resultados.

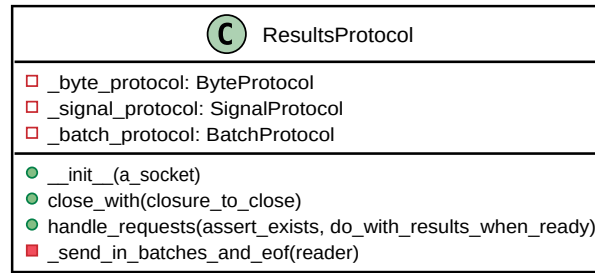


Figura 13: Diagrama de clase del protocolo del resultado

Batch Protocol: Protocolo de comunicación orientado al envío y recepción de grandes volúmenes de datos en forma de lotes (batches). Controla el tamaño máximo de cada lote y la cantidad total de elementos transmitidos, garantizando eficiencia y estabilidad en las transmisiones.

Signal Protocol: Protocolo de señalización utilizado para el intercambio de mensajes de control y propagación de errores. Permite la sincronización y la gestión de errores en la comunicación.

Byte Protocol: Protocolo base que define las operaciones elementales de envío y recepción de bytes, enteros y bloques de datos. Todos los protocolos de nivel superior se apoyan en él para garantizar una comunicación binaria confiable entre sockets.

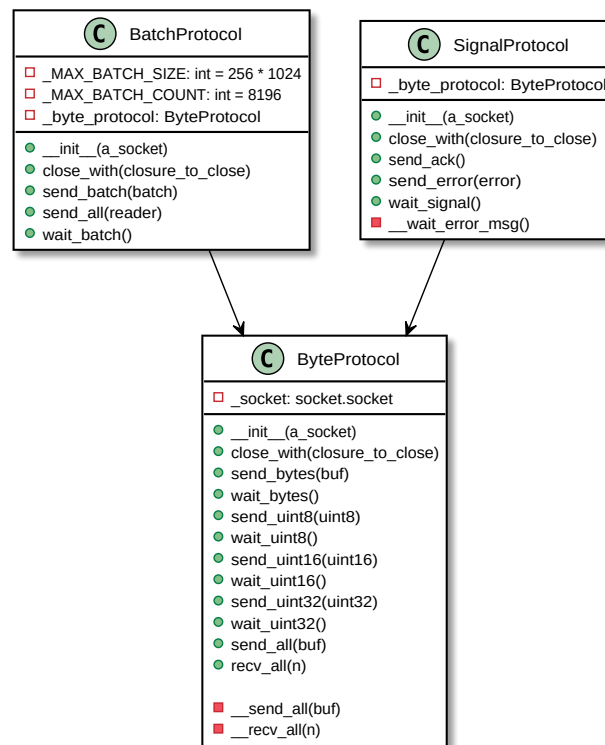


Figura 14: Diagrama de clases de los protocolos de bajo nivel

3.4. Tolerancia a fallos

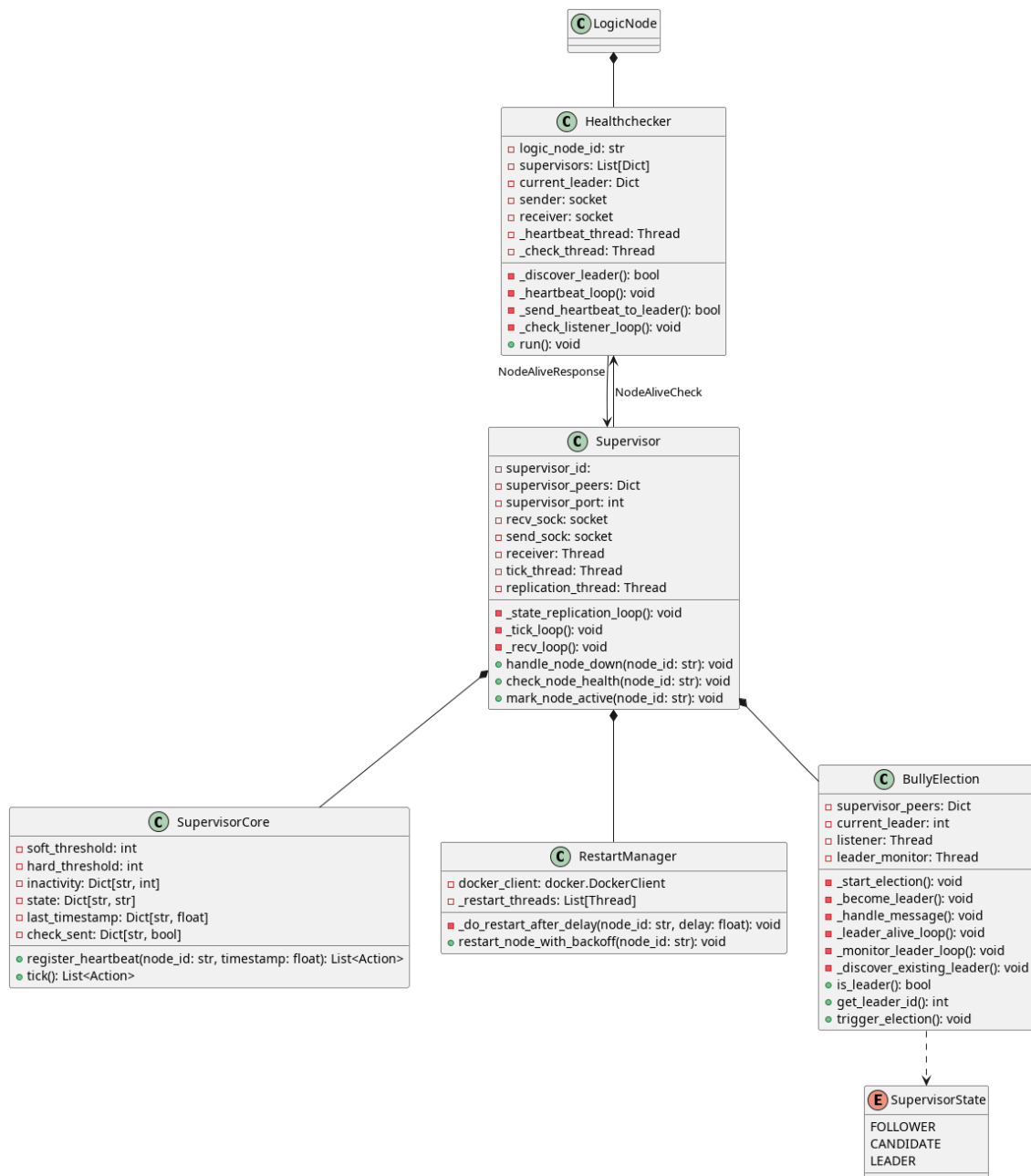


Figura 15: Diagrama de clases del Supervisor

El anterior diagrama presenta los componentes principales del mecanismo de tolerancia a fallos. Cada uno cumple un rol bien definido en el monitoreo de salud, detección de caídas, recuperación automática y coordinación entre supervisores.

Supervisor es la entidad responsable de monitorear los healthcheckers, detectar fallos y coordinar reinicios de nodos. Mantiene hilos dedicados a la recepción de heartbeats y al avance del tiempo lógico. Cuando hay más de un supervisor en ejecución, también gestiona la elección de líder.

SupervisorCore encapsula la lógica pura de detección de fallos:

- Estados por nodo: UP, SUSPECT, DOWN
- Contadores de inactividad (ticks sin heartbeat)

Devuelve acciones lógicas como `mark_up`, `send_check` y `mark_down`.

RestartManager es responsable de recuperar nodos caídos utilizando Docker. Aplica una política controlada de reinicios en hilos independientes y evita ciclos infinitos mediante una ventana deslizante de intentos.

BullyElection implementa el algoritmo Bully para garantizar que exista un único supervisor líder. El líder envía mensajes periódicos de `ALIVE`; si dejan de recibirse, se inicia una nueva elección considerando siempre como ganador al supervisor con ID mayor.

El **Healthchecker** corre en cada nodo de negocio y envía heartbeats al supervisor líder. Además, responde a verificaciones activas (`NODE_ALIVE_CHECK`) cuando se sospecha su caída. Si el líder deja de responder, ejecuta un proceso de descubrimiento del nuevo supervisor líder.

Dos protocolos soportan la comunicación:

- **HeartbeatProtocol**: intercambio de heartbeats y checks entre nodos y supervisor.
- **LeaderElectionProtocol**: mensajes del algoritmo Bully y consultas *¿Quién es el líder?*

Todos los mensajes son binarios para mayor eficiencia y compatibilidad con UDP.

En cuanto a la relación entre estos componentes, están acoplados de la siguiente forma:

- Supervisor compone a **SupervisorCore**, **RestartManager** y **BullyElection**.
- Cada nodo tiene un **Healthchecker** que se comunica únicamente con el supervisor líder.
- Las acciones emitidas por **SupervisorCore** pueden resultar en reinicios automáticos gestionados por **RestartManager**.

El resultado es un sistema modular, eficiente y resiliente frente a fallos.

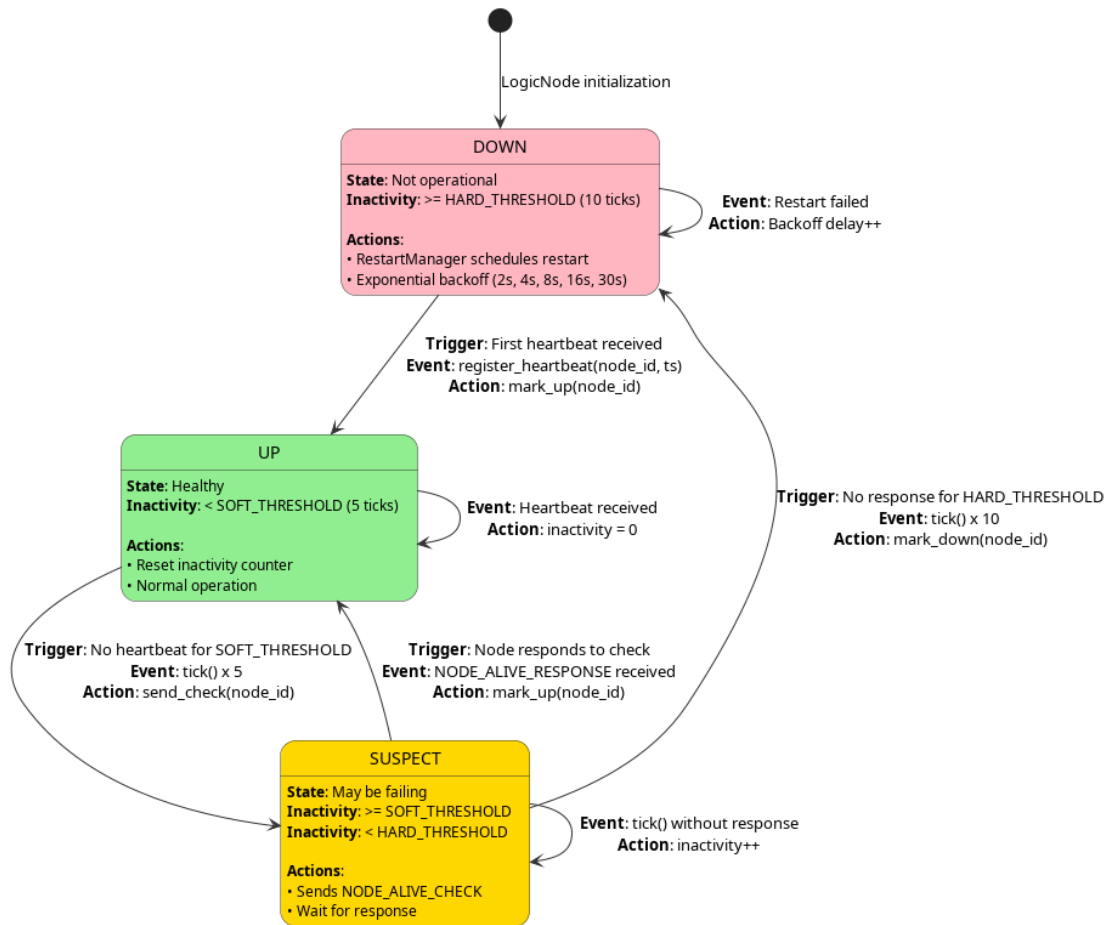


Figura 16: Diagrama de estados posibles de un nodo

El *Supervisor* líder mantiene, para cada nodo, una máquina de estados finitos (FSM) que modela su salud y decide cuándo sospechar una falla o declararlo caído. Sobre esta FSM se apoyan el envío de heartbeats, los checks activos y los reinicios automáticos.

Cada nodo puede estar en uno de tres estados:

- **DOWN:** El nodo se considera no operativo. Es el estado inicial y también el estado al que se pasa cuando se supera el umbral mas alto de inactividad (**HARD_THRESHOLD**). En este estado, el **RestartManager** puede programar reinicios controlados del contenedor.
- **UP:** El nodo está saludable y envía heartbeats periódicos al supervisor. Mientras el contador de inactividad se mantiene por debajo del umbral mas bajo (**SOFT_THRESHOLD**), el nodo permanece en este estado.
- **SUSPECT:** El supervisor sospecha que el nodo puede estar fallando. Se alcanza cuando el contador de inactividad se encuentra entre el mas alto y mas bajo. En este estado se disparan checks activos (**NODE_ALIVE_CHECK**) para confirmar si el nodo sigue vivo antes de marcarlo definitivamente como **DOWN**.

Las transiciones entre estos estados se basan en heartbeats recibidos y en el número de ticks sin actividad:

- **DOWN → UP:** Primer **heartbeat** válido recibido para ese nodo. Se resetea el contador de inactividad y se marca el nodo como operativo (**mark_up**).

- **UP → SUSPECT**: No se reciben heartbeats durante `SOFT_THRESHOLD` ticks. La llamada periódica a `tick()` incrementa el contador y, al cruzar el umbral mas bajo, se emite una acción `send_check` para enviar un `NODE_ALIVE_CHECK`.
- **SUSPECT → UP**: El nodo responde (directa o indirectamente) con un `NODE_ALIVE_RESPONSE`. Se resetea la inactividad y se limpia el estado sospechoso (`mark_up`).
- **SUSPECT → DOWN**: No hay respuesta dentro de `HARD_THRESHOLD` ticks totales. `tick()` cruza el umbral mas alto y se emite `mark_down`, que puede desencadenar un reinicio vía `RestartManager`.

Algunos parámetros clave que definen la FSM son:

- `HEARTBEAT_TICK`: intervalo entre ejecuciones de `tick()`.
- `SOFT_THRESHOLD`: número de ticks sin heartbeat a partir del cual el nodo pasa a estado `SUSPECT`.
- `HARD_THRESHOLD`: número de ticks sin heartbeat a partir del cual el nodo pasa a estado `DOWN`.
- `heartbeat_interval`: intervalo de envío de heartbeats por parte de los nodos.
- Parámetros de reinicio (`MAX_RESTART_ATTEMPTS`, `RESTART_WINDOW`) que limitan y espacia los reinicios cuando un nodo permanece en `DOWN`.

En conjunto, esta máquina de estados permite distinguir entre fallos transitorios (resueltos en estado `SUSPECT`) y caídas reales (estado `DOWN`), integrando detección, validación y recuperación automática de nodos.

4. Vista de Procesos

La planificación de las consultas se diseñó con el objetivo de reducir el volumen de datos lo antes posible. Para ello, se aplican en primer lugar las proyecciones, eliminando las columnas irrelevantes, y los selects, descartando las filas que no cumplen los criterios de filtrado. Este enfoque no solo optimiza la ejecución al minimizar la cantidad de datos procesados en las etapas posteriores, sino que además reduce la complejidad de operaciones más costosas como group by y join.

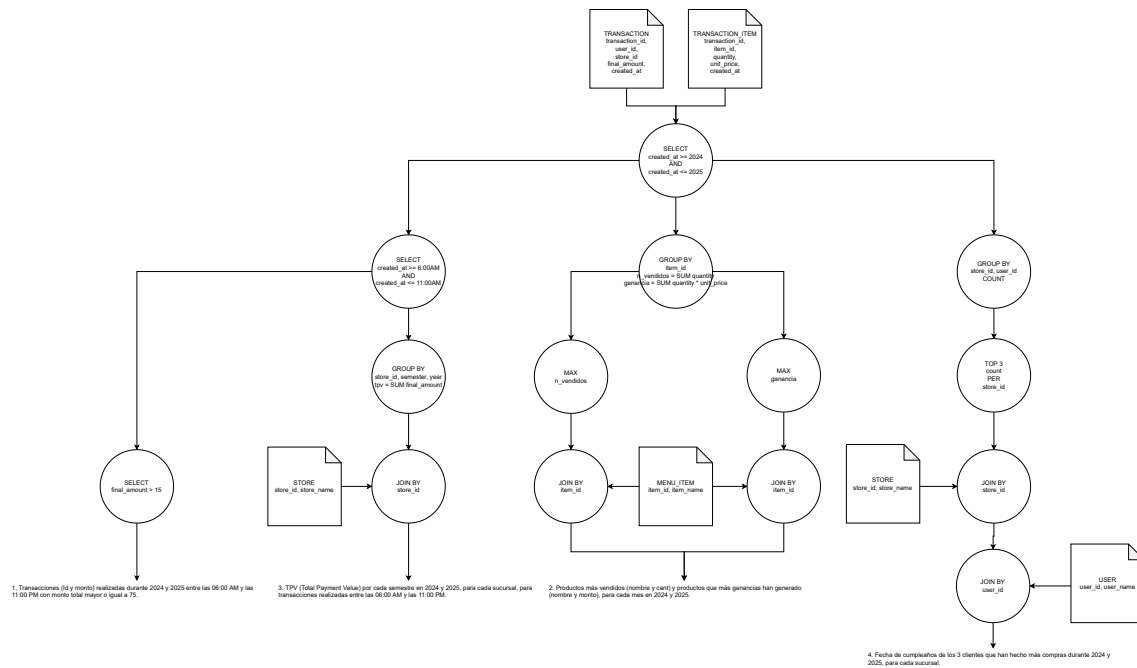


Figura 17: Diagrama DAG: Planificación de consultas

Ciertas tareas se pueden realizar en paralelo, por ejemplo la selección puede dividirse entre varios nodos ya que no es una operación que requiere de la totalidad de los datos para resolverse, a continuación se muestra como dos nodos de selección trabajan en paralelo.

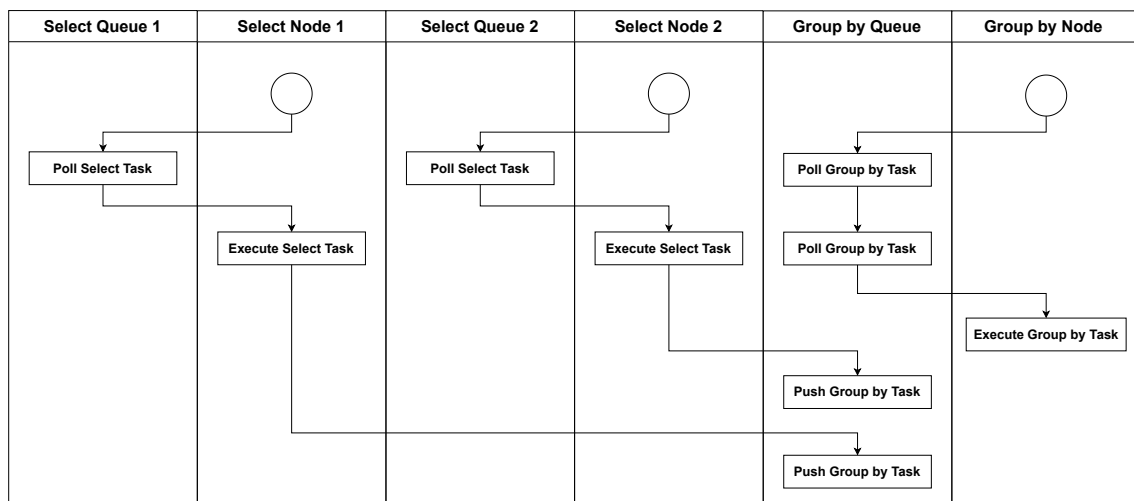


Figura 18: Diagrama de actividades: Selección paralela

Este diagrama describe cómo un Select Node procesa las tareas que recibe a través del middleware.

El `SelectNode` consume las tareas desde la cola correspondiente utilizando `consume_select_tasks`. Cada tarea es derivada a un `TypeHandler`, que se obtiene con `get_handler(task.type_query)`. El handler ejecuta la operación (`exec(task.data)`), produciendo un resultado parcial (`task_output`). El resultado se envía al `Middleware` mediante `send_output`, que utiliza un esquema de hashing (`hash(key)`) para determinar a qué `GroupQueue` debe ir. Finalmente, el `GroupNode` correspondiente recibe la tarea y ejecuta `handle_group(task)`, continuando con la siguiente etapa del pipeline.

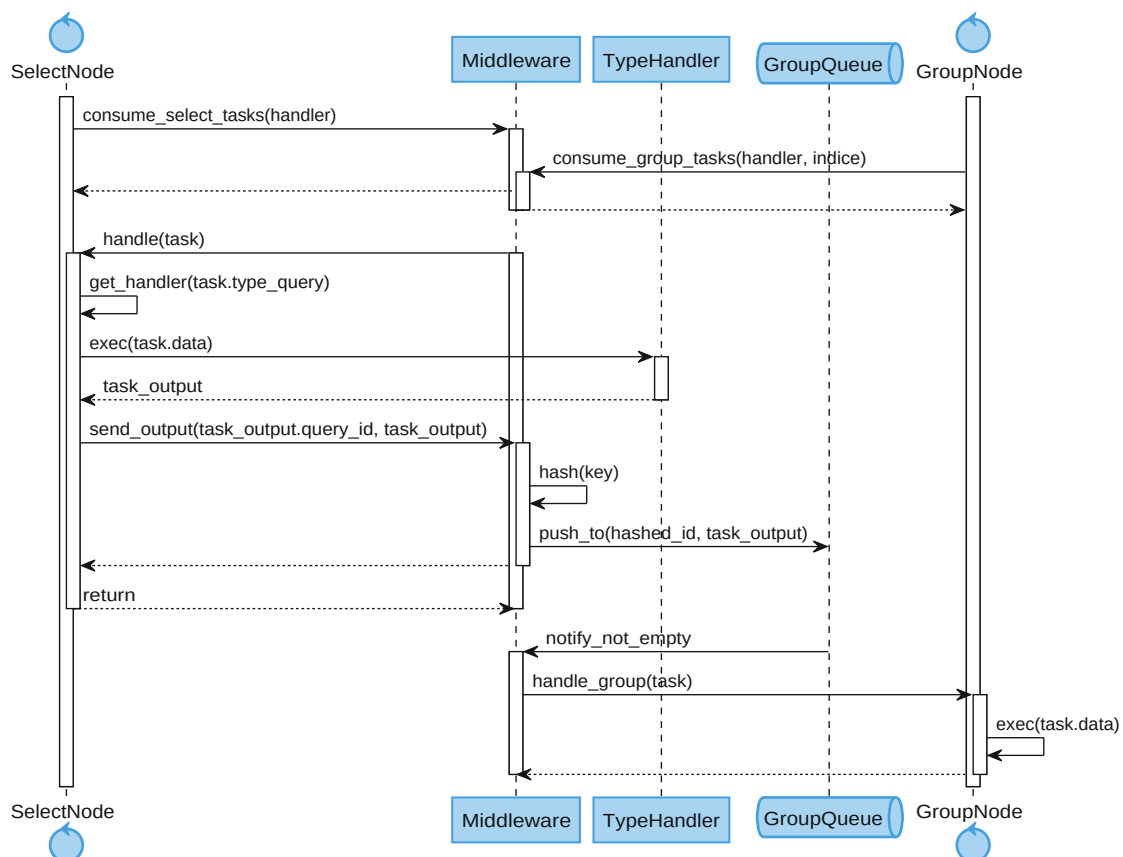


Figura 19: Diagrama de secuencia: Select Node

Este diagrama describe cómo un Group Node procesa las tareas que recibe a través del middleware.

El GroupNode recibe tareas desde el middleware (handle(task)). Se obtiene el handler correspondiente mediante get_handler(task.type_query). El handler ejecuta la lógica de agrupación (exec(task.data)), generando resultados junto con una clave de partición (task_output, hash_key). El resultado se envía al middleware con send_output, que utiliza hash(hash_key) para decidir a qué JoinQueue corresponde. El JoinNode recibe los resultados parciales y continúa con las uniones necesarias.

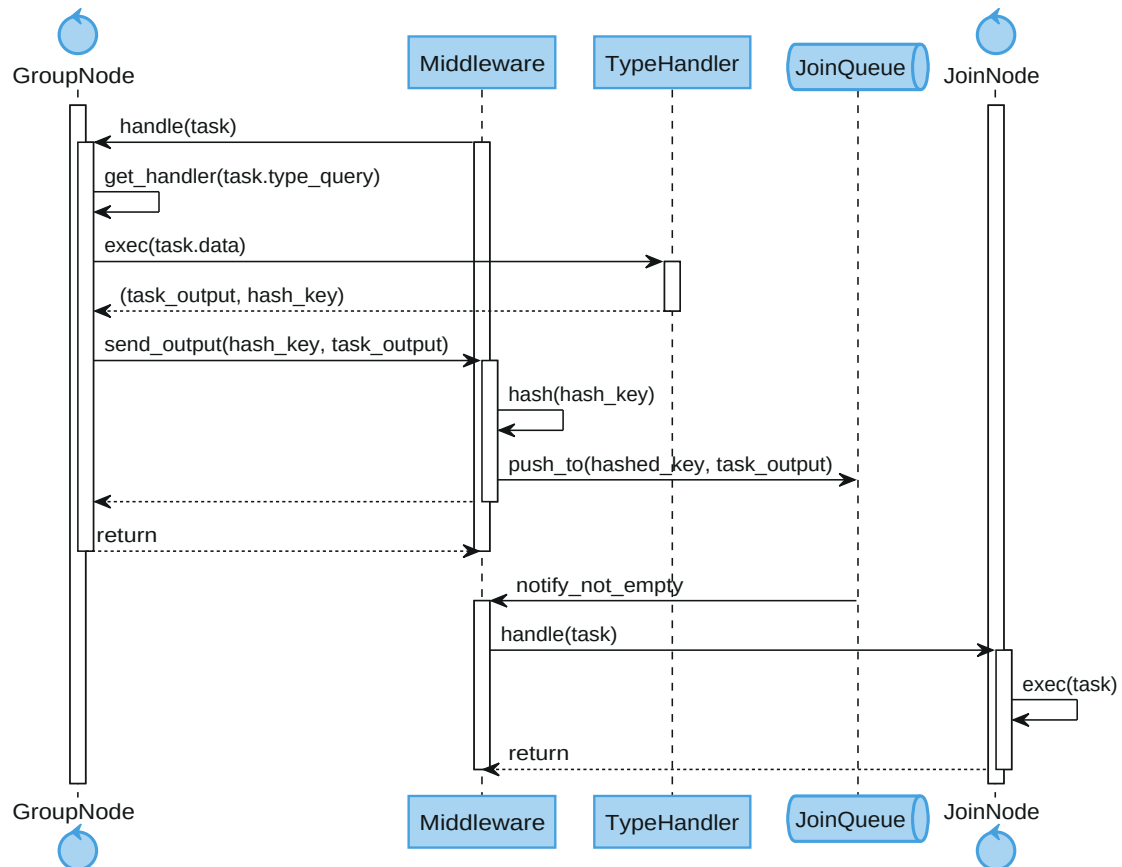


Figura 20: Diagrama de secuencia: Group Node

4.1. State Storage

Los diferentes elementos que componen el sistema podrían verse afectados por un fallo en cualquier momento de su operación, por lo que podrían quedar ellos mismo o todo el sistema en un estado inconsistente.

En el siguiente diagrama se muestra como se utiliza el **State Storage** para asegurar la resiliencia a fallos y la consistencia de los datos ya procesados por el **Groupby Node**.

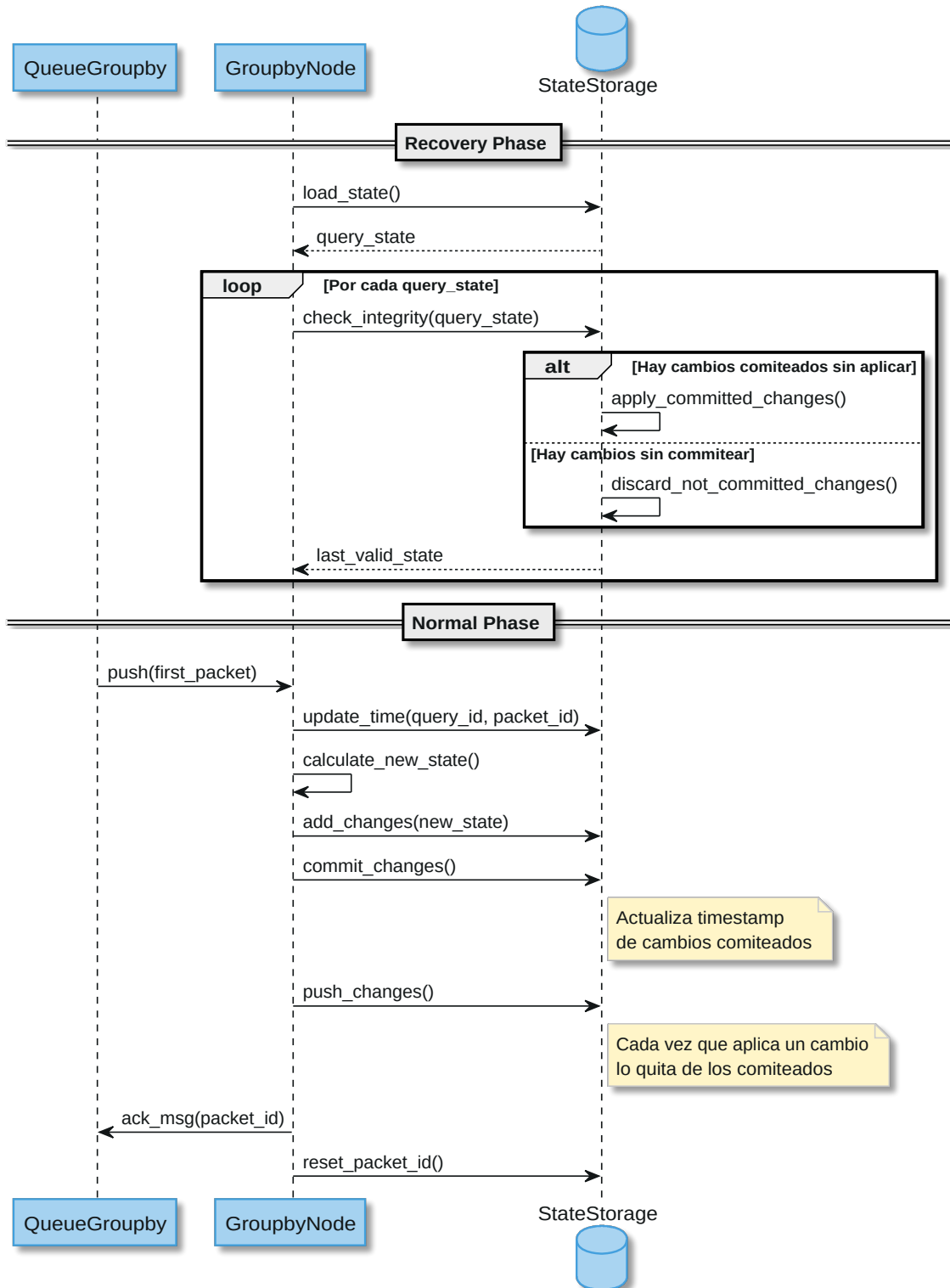


Figura 21: Diagrama de secuencia: Interacción entre Groupby Node y su State Storage

Fase de Recuperación

- **Carga del Estado:** Se recupera el estado persistido que incluye:
 - Estado actual de las queries
 - Timestamp del último cambio aplicado
 - Timestamp del último cambio comprometido
 - ID del paquete en proceso
 - Metadatos y tags adicionales
- **Verificación de Integridad:** Para cada query almacenada:
 - Si hay cambios comprometidos pendientes → Se aplican automáticamente
 - Si hay cambios sin comprometer → Se descartan para mantener consistencia
 - Se retorna el último estado válido

Fase de Procesamiento Normal

- **Registro del Nuevo Paquete:**
 - Se actualiza el timestamp del cambio actual
 - Se almacena la información del paquete (query_id, packet_id, metadatos)
- **Cálculo del Nuevo Estado:**
 - Se procesa la lógica de negocio con los datos del paquete
 - Se genera la nueva versión/cambios a aplicar
- **Preparación de Cambios:**
 - Se escriben los cambios en la sección temporal de cambios pendientes
- **Compromiso de Cambios (commit_changes):**
 - Se incrementa el timestamp de cambios comprometidos
 - Los cambios pasan de “pendientes” a “comprometidos” (atomicidad)
- **Aplicación de Cambios (push_changes):**
 - Se fusionan los cambios comprometidos con el estado actual
 - Se eliminan los cambios ya aplicados de la cola de comprometidos
- **Finalización del Procesamiento:**
 - Se envía ACK al mensaje para confirmar procesamiento exitoso
 - Se limpia la información del paquete actual (reset_packet_id)

Notese que, si bien el ejemplo utiliza el caso particular del **Groupby Node**, la lógica del **State Storage** no está acoplada a un estado específico. En su lugar, emplea una interfaz de estado que define los mecanismos de serialización y deserialización.

Esto permite que el **State Storage** sea utilizado por distintos componentes del sistema, cada uno declarando el estado que necesita persistir según la función que desempeña.

4.2. Eleccion de Lider: Bully

El algoritmo Bully asegura que el supervisor con mayor identificador actúe como líder del sistema. La siguiente secuencia ilustra el caso donde **Supervisor2** (ID=2) se reintegra mientras **Supervisor1** (ID=1) está funcionando como líder, y cómo se realiza la transición de liderazgo sin perder monitoreo ni estado.

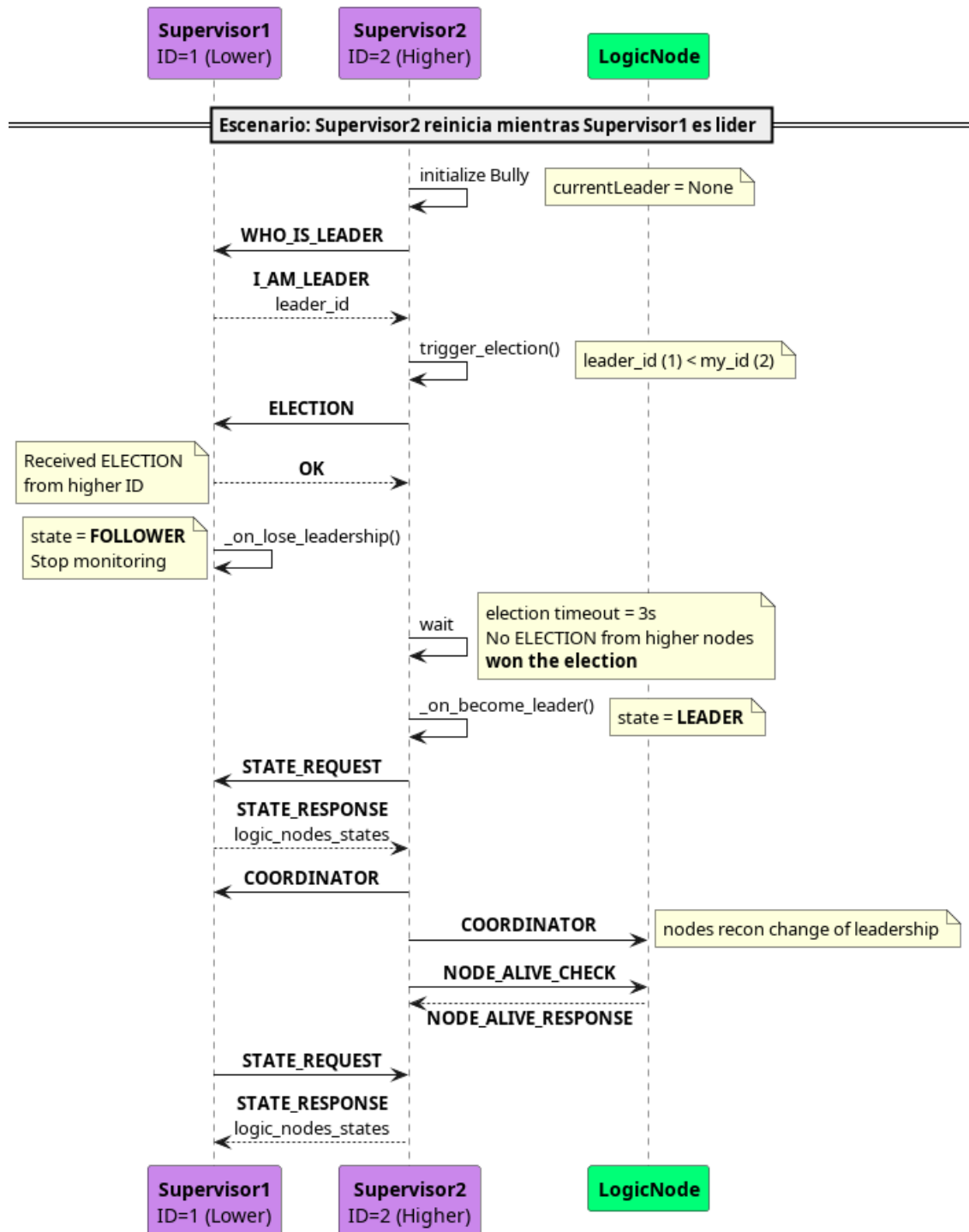


Figura 22: Diagrama de secuencia: Ingresar un Supervisor de mayor ID que el líder

Hay dos supervisores:

- **Supervisor1:** ID = 1 (prioridad menor), inicialmente líder.
- **Supervisor2:** ID = 2 (prioridad mayor), se reinicia y vuelve al cluster.

Al arrancar, Supervisor2 no asume nada y comienza en estado **FOLLOWER**.

Antes de disparar una elección, Supervisor2 pregunta quién es el líder:

- Envía **WHO_IS_LEADER** a sus peers.
- Supervisor1 responde indicando que es el líder actual.

Con esa información, Supervisor2 compara IDs: si el **leader_id** recibido es menor que su propio ID, decide desafiar al líder y dispara el algoritmo Bully; si fuera mayor o igual, simplemente acepta al líder existente.

Cuando Supervisor2 decide desafiar:

- Inicia una elección (**trigger_election()**), enviando **ELECTION** a supervisores con ID mayor (en este caso no hay).
- Supervisor1, al recibir **ELECTION** desde un ID mayor, responde y deja de ser líder (pasa a **FOLLOWER** y desactiva su monitoreo).
- Tras un timeout sin respuestas de IDs mayores, Supervisor2 se proclama líder.

Al convertirse en líder, Supervisor2:

- Cambia su estado a **LEADER** y activa el monitoreo de nodos.
- Recupera el estado de los nodos, ya sea desde el líder anterior o usando el estado replicado.
- Marca los nodos heredados como candidatos a chequear (pasando temporalmente a **SUSPECT**) y les envía checks activos para confirmar que siguen vivos.

De esta forma se evita asumir ciegamente un estado desactualizado y se revalida rápidamente qué nodos están realmente operativos.

Finalmente, el nuevo líder:

- Anuncia su liderazgo mediante mensajes de **COORDINATOR** a otros supervisores.
- Continúa replicando periódicamente el estado para que un futuro cambio de líder pueda reutilizar esa información.

Este protocolo garantiza que:

- Nunca haya dos líderes simultáneos (el líder anterior se desactiva antes de que el nuevo comience a monitorear).
- El líder con mayor ID sea quien coordina, siempre que esté disponible.
- Un supervisor que se reinicia pueda integrarse sin romper el servicio, desafiando al líder actual sólo cuando corresponde.
- La transición de liderazgo preserve el estado de los nodos y lo revalide rápidamente, manteniendo la funcionalidad de monitoreo y recuperación ante fallos.

En la siguiente secuencia quisimos detallar qué ocurre cuando el *supervisor líder* se cae y cómo el sistema recupera automáticamente un nuevo líder usando el algoritmo Bully.

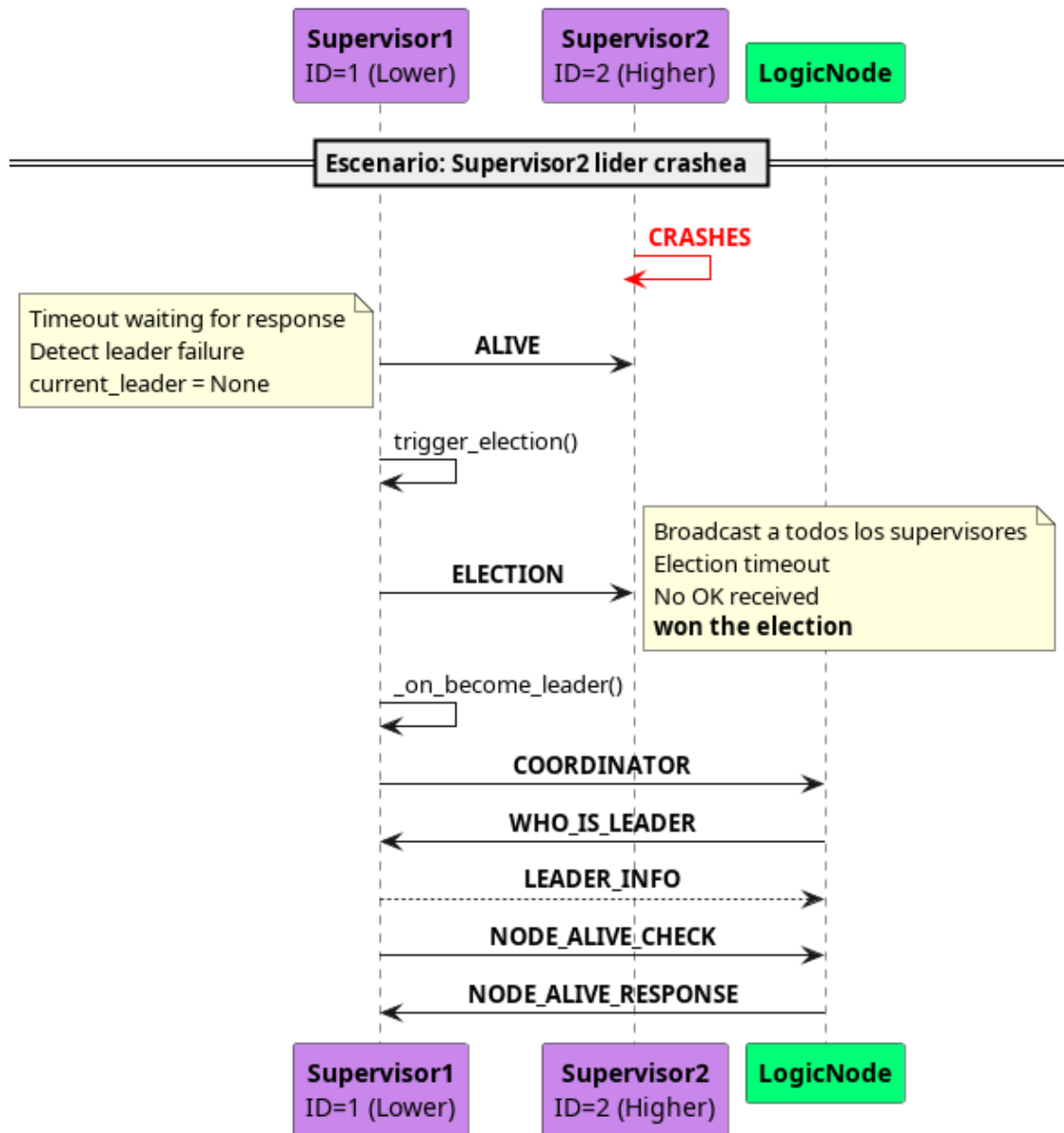


Figura 23: Diagrama de secuencia: Cambio de lider por caída

En el estado inicial hay:

- Un **supervisor líder** (ID más alto) que monitorea los nodos.
- Un **supervisor followers** que conoce quién es el líder actual.
- **LogicNodes** que envían heartbeats al líder.

El follower ejecuta un bucle de monitoreo del líder. Periódicamente verifica si sigue recibiendo señales de vida del líder (mensajes **ALIVE** o equivalente). Si durante un intervalo configurado no llega ninguna señal, el follower asume que el líder ha fallado y limpia **current_leader**.

Al detectar el fallo, el follower dispara **trigger_election()**:

- Envía mensajes **ELECTION** a todos los supervisores con ID mayor.
- Si alguno responde, ese supervisor de mayor ID continuará la elección y eventualmente se proclamará líder.
- Si nadie responde (todos los IDs mayores están caídos), el follower gana por defecto.

Tras un **election_timeout** sin respuestas de IDs mayores, el supervisor que inició la elección se convierte en nuevo líder.

Cuando un supervisor se proclama líder:

- Cambia su estado interno a **LEADER** y actualiza **current_leader**.
- Ejecuta un callback en **Supervisor** (**on_become_leader()**) para activar el monitoreo de nodos.
- A partir de este momento, comienza a enviar mensajes de **ALIVE** a los demás supervisores para que sepan quién es el líder actual.

Los **LogicNode** siguen enviando heartbeats al líder anterior hasta que dejan de recibir respuestas. Cuando detectan que el líder no responde:

- Ejecutan un proceso de descubrimiento (**_discover_leader()**).
- Envían **WHO_IS_LEADER** a los supervisores.
- El nuevo líder responde con **LEADER.INFO**, indicando su identidad y dirección.
- Los Nodos actualizan su **current_leader** y redirigen los siguientes heartbeats al nuevo líder.

El diagrama de secuencia muestra cómo:

1. El fallo del líder se detecta automáticamente por timeout.
2. El algoritmo Bully elige un nuevo líder.
3. Los Nodos redescubren al líder y retoman el envío de heartbeats.

De esta forma, el sistema mantiene la funcionalidad de monitoreo y tolerancia a fallos incluso ante la caída completa del supervisor líder.

4.3. Protocolo de comunicacion entre Healthchecker y Supervisor

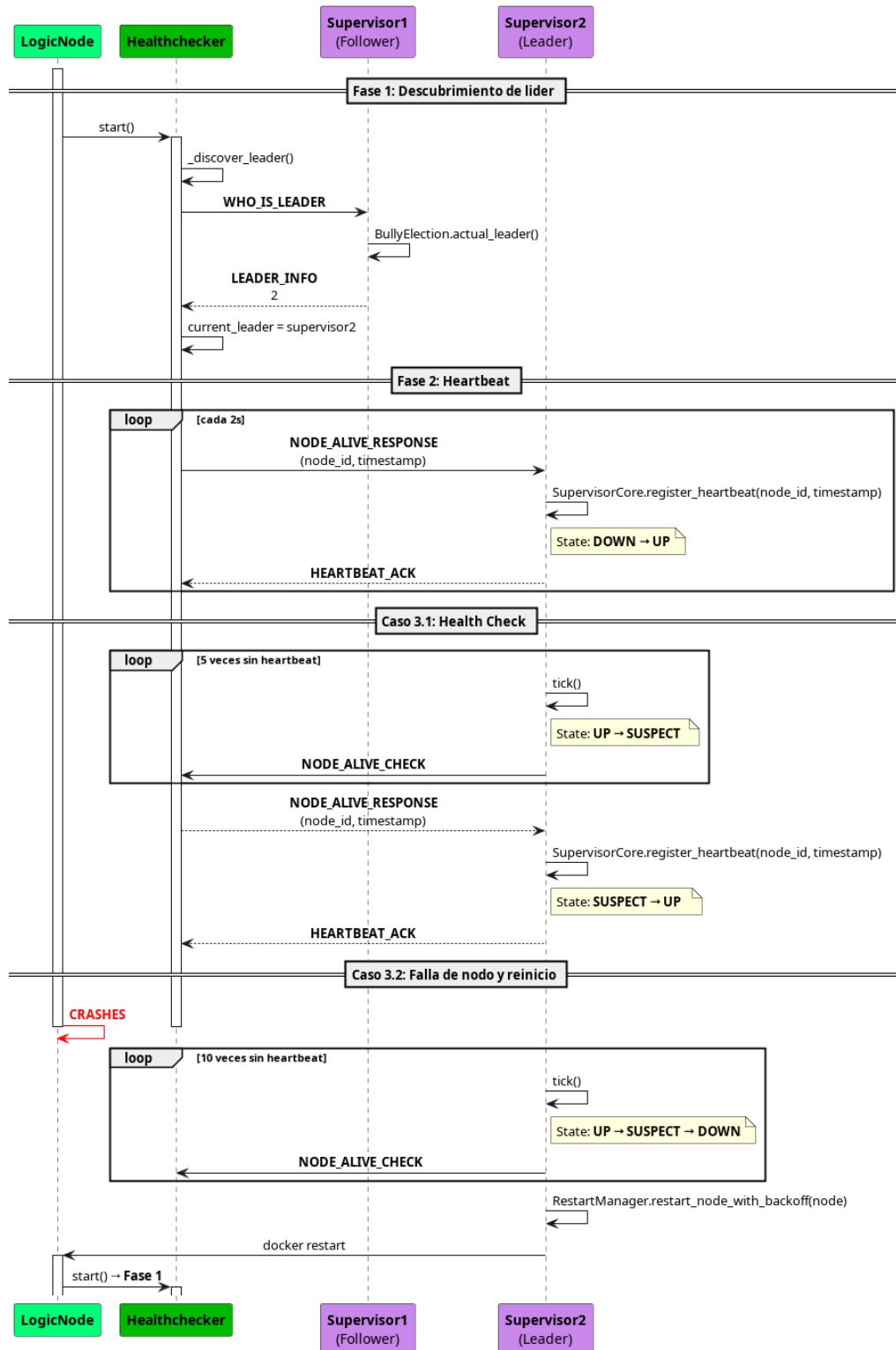


Figura 24: Diagrama de secuencia: Protocolo de Healthchecker y Supervisor

El anterior diagrama describe cómo el **Healthchecker** de cada nodo se comunica con el **Supervisor líder** para monitorear disponibilidad y recuperar fallos.

Cuando un nodo inicia, no sabe quién es el líder. El Healthchecker envía `WHO_IS_LEADER` y un supervisor responde `LEADER_INFO`. Esto permite:

- Redirigir heartbeats al supervisor correcto.
- Evitar elecciones innecesarias.

Una vez que conoce al líder, el Healthchecker envía periódicamente `NODE_ALIVE_RESPONSE`. El líder:

- Registra la actividad del nodo y lo marca `UP`.
- Responde con `HEARTBEAT_ACK`.

Si el líder deja de recibir heartbeats por un período configurado, el nodo pasa a `SUSPECT`. Para descartar falsos positivos, el supervisor envía `NODE_ALIVE_CHECK`:

- Si el worker responde, vuelve a `UP` sin reinicios.
- Se evita una recuperación innecesaria.

Si el nodo no responde dentro del umbral mas alto:

- Se marca como `DOWN`.
- El **RestartManager** reinicia su contenedor.
- Cuando el nodo vuelve, repite la fase de **Descubrimiento de líder** y se reincorpora automáticamente.

Este protocolo combina:

- Detección rápida de fallos reales.
- Protección ante falsos positivos.
- Recuperación automática.

Asegurando que el sistema mantenga siempre monitoreo activo y disponibilidad del servicio aún frente a fallos de nodos.

5. Vista de Desarrollo

Business Logic: Contiene la lógica de negocio asociada a las consultas definidas en el alcance. Aquí se implementan las reglas para filtrar, agrupar, unir y consolidar datos, de manera independiente de la infraestructura subyacente.

Processing Nodes: Incluye las implementaciones de los distintos nodos de procesamiento (Select, Group, Join, Result). Cada nodo ejecuta tareas específicas del pipeline y se comunica con otros a través del middleware.

Type Handlers: Define cómo se va a comportar un nodo en función de la consulta que se está ejecutando en él.

Middleware: Proporciona la abstracción de comunicación entre nodos mediante un modelo orientado a colas. Se asegura de la entrega confiable de mensajes, la asincronía y la tolerancia a fallos.

State Storage: Almacenamiento persistente para partes del sistema con estado. Garantiza que los datos parciales se conserven en caso de falla.

Result Storage: Repositorio persistente para almacenar los resultados finales, asociados a clientes mediante el client_id.

Routing: Encargado de dirigir cada mensaje hacia el nodo o cola correspondiente, aplicando estrategias de hash y balanceo de carga.

Business Protocol: Define el contrato de comunicación entre cliente y servidor (registro de consultas, consulta de resultados, manejo de identificadores).

Task Protocol: Describe cómo se representan, envían y reciben las tareas de procesamiento entre nodos. Incluye metadatos como tipo de consulta, identificador de cliente y partición.

Data Protocol: Normaliza el formato de los datos transaccionales y de negocio (transacciones, usuarios, productos, sucursales) que circulan en el sistema.

Byte Protocol: Capa más baja de comunicación, que define la serialización y deserialización de los mensajes para su envío eficiente a través del middleware.

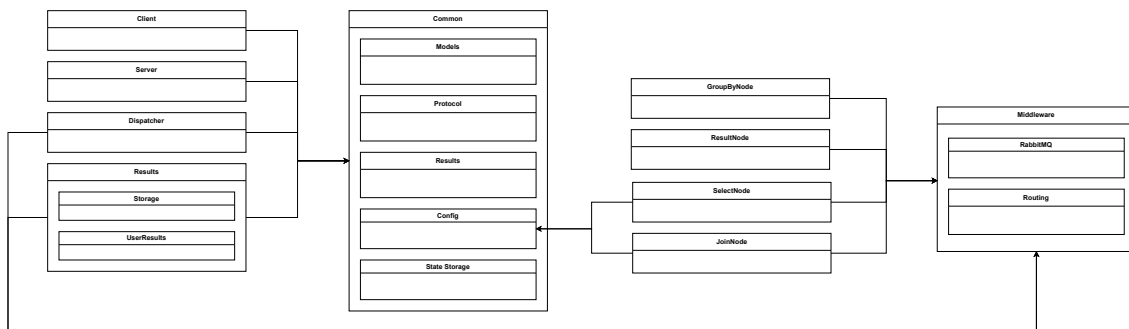


Figura 25: Diagrama de paquetes

Dentro del paquete de protocolos se definen los protocolos principales, homónimos de las entidades que los utilizan (`Client`, `Server`, etc.). Estos protocolos están implementados siguiendo un patrón Composite, en el cual los protocolos de entidad se componen de protocolos de un nivel intermedio, llamados Batch y Signal. A su vez, estos protocolos intermedios se implementan sobre un protocolo de nivel inferior, denominado `ByteProtocol`.

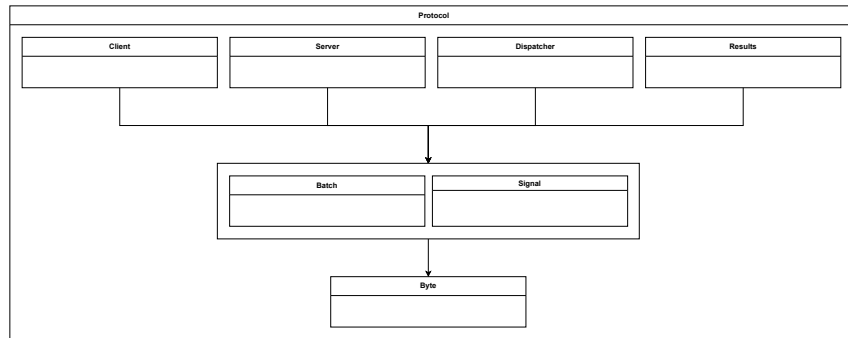


Figura 26: Paquete de protocolos

El paquete Models contiene las clases que representan las distintas entidades del dominio de negocio, mientras que el paquete Results agrupa las estructuras correspondientes a los resultados de las consultas procesadas.

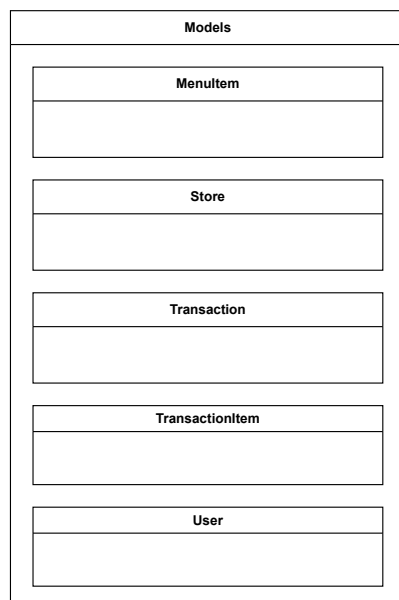


Figura 27: Paquete de modelos

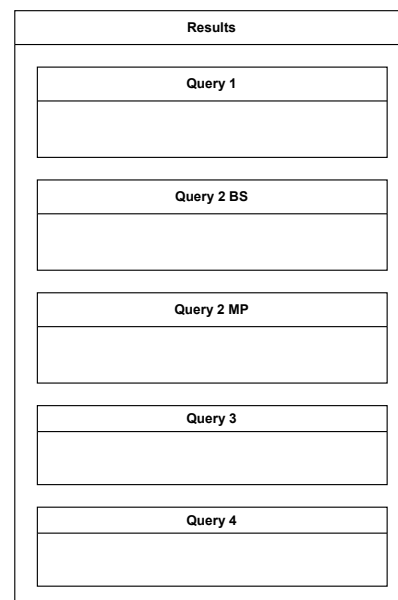


Figura 28: Paquete de resultados

6. Vista Física

6.1. Topología compartida

La **topología del sistema** se define a partir de los distintos pipelines necesarios para resolver cada consulta. Todas las consultas comparten una sección común formada por tres componentes principales.

Servidor: Actúa como balanceador de carga: cuando un usuario envía una consulta, lo conecta con un Dispatcher; en cambio, cuando el usuario solicita acceder a los resultados, el servidor se encarga de lo conecta con su Data Storage de Resultados.

Resultados: Los resultados de cada consulta se almacenan en disco y se asocian al ID del cliente que la originó, lo que permite acceder a ellos de manera asincrónica en cualquier momento posterior.

Dispatcher: recibe los datos, aplica las validaciones necesarias y transforma a un formato conveniente. Además, se encarga de reenviar los datos a través de una cola de tareas para su procesamiento.

6.2. Topología entre consultas

Para explicar la topología del sistema, la organizamos en función de los distintos pipelines asociados a cada consulta. Sin embargo, es importante destacar que como muchos de los datos iniciales y de los resultados parciales son compartidos por múltiples consultas. Para optimizar la resolución y evitar cálculos redundantes, los nodos pueden retransmitir sus resultados a varios pipelines de manera simultánea, lo que permite que diferentes consultas reutilicen la misma información.

6.3. Topología por consulta

En la consulta 1, cada **Select Node** es capaz de ejecutar las tres selecciones requeridas. Para acelerar la resolución, es posible utilizar múltiples nodos en paralelo. Los resultados generados se envían a un nodo de resultados **Result Node** donde se consolidan y se van retransmitiendo al almacenamiento persistente de resultados.

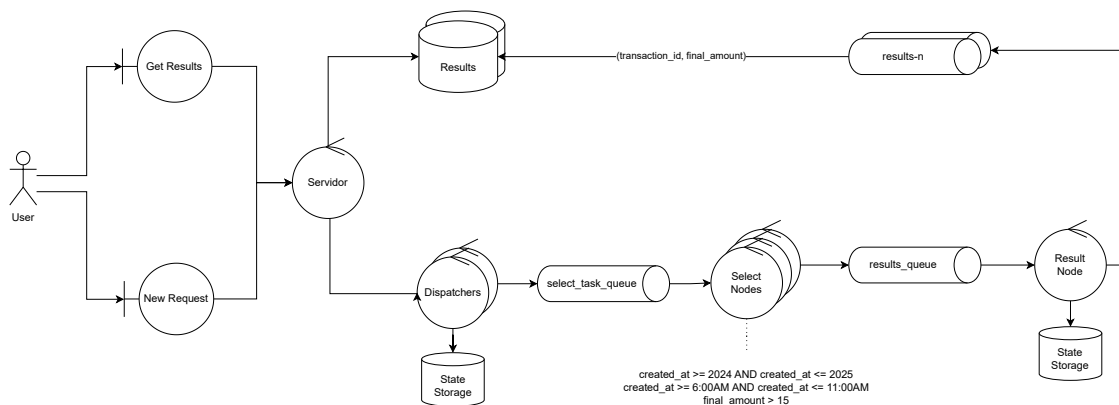


Figura 29: Diagrama de robustez consulta 1°

En la consulta 2, cada **Select Node** únicamente realiza la primera selección por año. Posteriormente, los resultados se dirigen a un **Group Node**, el cual calcula dos métricas: el producto más vendido y el de mayor ganancia. Estos datos se bifurcan en dos pipelines independientes, que

realizan un join y consolidan la información en el **Result Node**. Una vez unificados, los datos se almacenan de forma persistente, del mismo modo que en la consulta 1.

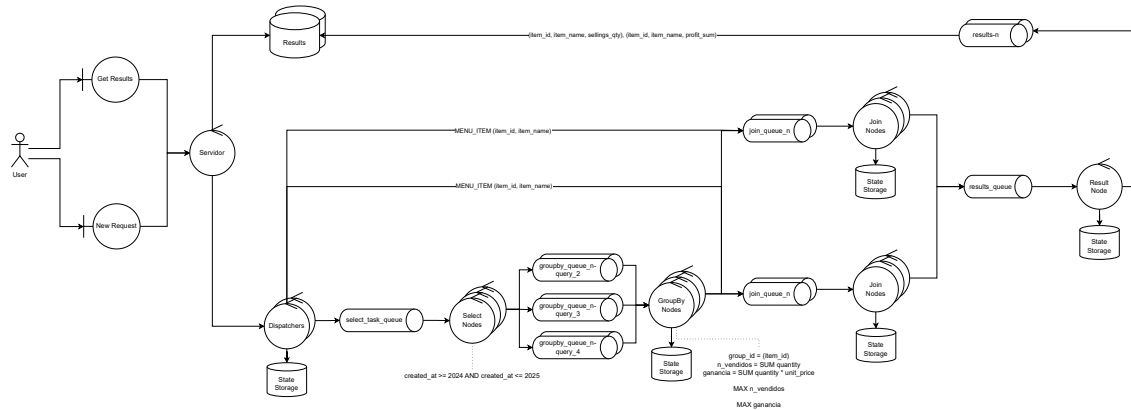


Figura 30: Diagrama de robustez consulta 2°

En la consulta 3 se reutiliza el resultado parcial de la consulta 1, correspondiente a la selección por año y hora. A continuación, el **Group Node** agrupa los datos y calcula el TPV. Finalmente, se realiza un join, tras el cual los resultados se consolidan y se almacenan en disco.

Es importante destacar que, como se mencionó previamente, las consultas 1 y 3 utilizan los mismos datos en su etapa inicial (la selección por año). Aunque se representen en gráficos separados, este cálculo se realiza una sola vez y se retransmite a múltiples pipelines.

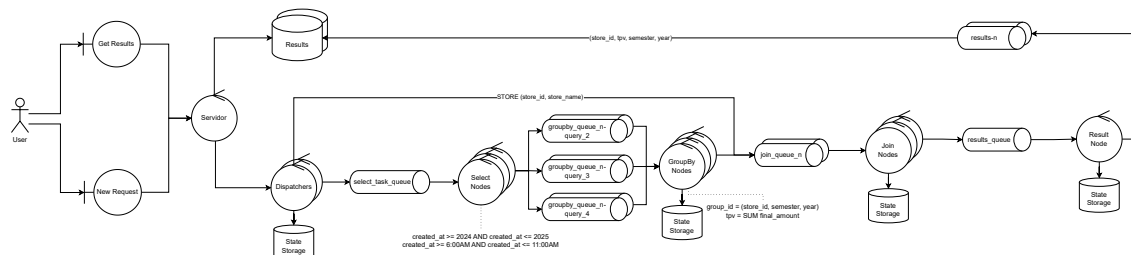


Figura 31: Diagrama de robustez consulta 3°

En la consulta 4 también se reutiliza el resultado parcial de la consulta 1, correspondiente a la selección por año. El **Group Node** ejecuta los cálculos requeridos por la consulta, en este caso, la obtención del top 3 de clientes con más compras. Finalmente, se realizan dos joins consecutivos y, al igual que en los casos anteriores, los datos se consolidan y se almacenan en disco.

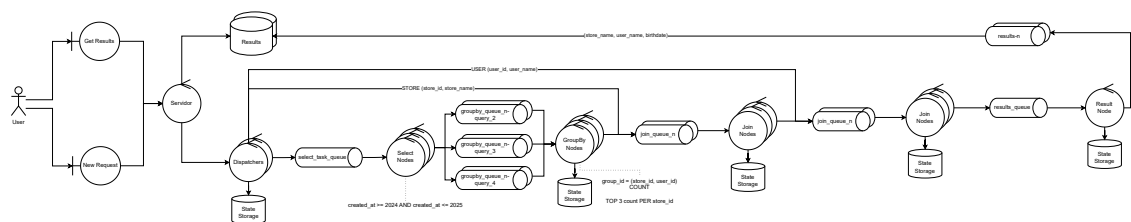


Figura 32: Diagrama de robustez consulta 4°

6.4. Despliegue

El **Servidor** y el **Middleware** se despliegan como componentes independientes.

Para aumentar la capacidad de procesamiento de solicitudes concurrentes, se permite el despliegue de múltiples instancias de **Dispatcher** y para aumentar la capacidad de almacenamiento, se permite el despliegue de múltiples instancias de **Result Storage**.

La **Select Task Queues** y los **Select Nodes** se despliegan de manera desacoplada, dado que cualquier nodo puede consumir tareas de la cola.

En contraste, los nodos que requieren mantener estado, como **Join**, **Group** y **Result Node**, deben desplegarse junto con su cola y su almacenamiento asociados, a fin de preservar la coherencia del procesamiento.

En este caso, la asociación estricta con una cola garantiza la consistencia en la asignación de consultas a nodos específicos. De forma similar, cada **Result Storage** se despliega junto con su cola asociada y la asociación estricta con un usuario mediante el hash de su id, permite asociar cada usuario con sus resultados.

Finalmente los **Supervisor** se despliegan con copias redundantes pero con un líder único para garantizar disponibilidad y al mismo tiempo consistencia.

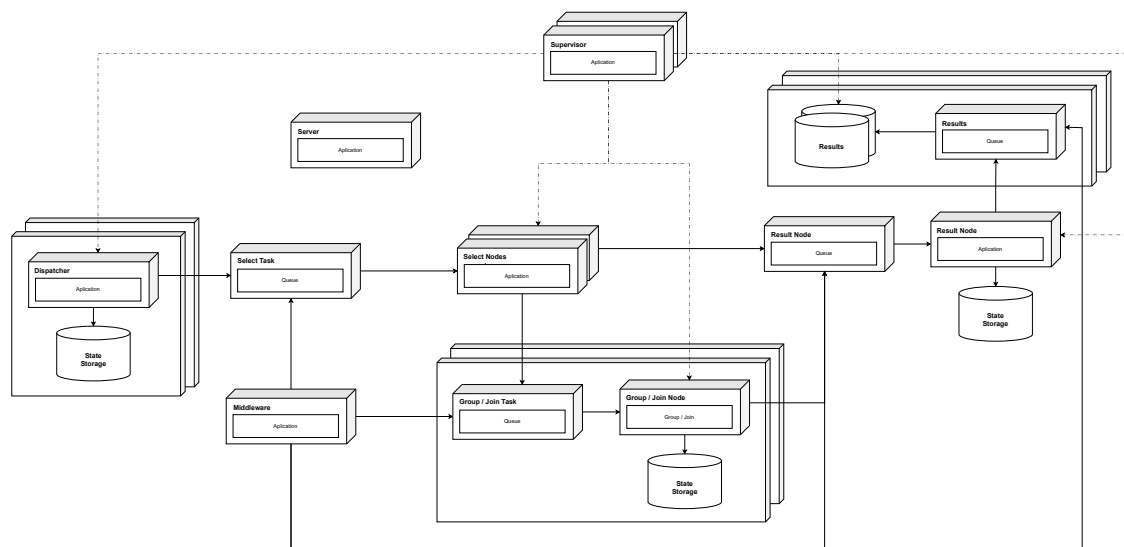


Figura 33: Diagrama de despliegue